Ex. No: 7

Title: Perceptron and Forward Propagation

Packages: NumPy, Matplotlib

AIM:

To implement a single perceptron with forward propagation and train it to learn the AND gate.

ALGORITHM:

1. Start the program and import NumPy and Matplotlib.
2. Take the AND gate inputs X and targets y.
3. Initialize the weights, bias and learning rate.
4. Forward propagation: calculate z = w.x + b and apply the step function.
5. Find the error = target - output.
6. Update the weights and bias: w = w + lr x error x input, b = b + lr x error.
7. Repeat steps 4 to 6 for all inputs for 10 epochs.
8. Print the weights, bias and outputs, and plot the decision boundary.
9. Display the output and stop.

PROGRAM:

```
import numpy as np
import matplotlib.pyplot as plt

# AND gate data
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
y = np.array([0, 0, 0, 1])

w = np.zeros(2)
b = 0.0
lr = 0.1

def step(z):
    return 1 if z >= 0 else 0

# Training the perceptron
for epoch in range(10):
    for xi, target in zip(X, y):
        output = step(np.dot(w, xi) + b)      # forward propagation
        error = target - output
        w = w + lr * error * xi               # weight update
        b = b + lr * error

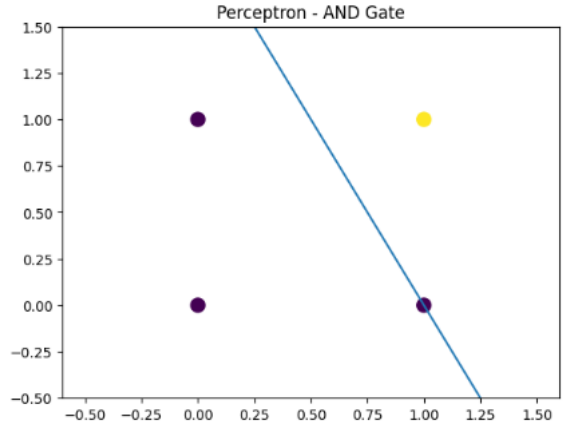
print("Weights =", w)
print("Bias =", round(b, 2))
for xi in X:
    print("Input", xi, "-> Output", step(np.dot(w, xi) + b))

# Plot decision boundary
plt.scatter(X[:, 0], X[:, 1], c=y, s=100)
xs = np.linspace(-0.5, 1.5, 10)
plt.plot(xs, -(w[0] * xs + b) / w[1])
plt.ylim(-0.5, 1.5)
```

```
plt.title("Perceptron - AND Gate")
plt.show()
```

OUTPUT:

```
Weights = [0.2 0.1]
Bias = -0.2
Input [0 0] -> Output 0
Input [0 1] -> Output 0
Input [1 0] -> Output 0
Input [1 1] -> Output 1
```



RESULT:

The perceptron was trained successfully and it produced the correct output for the AND gate.